CS2013: Programación III

Teoría: DFS y BFS

José Chávez

Agenda

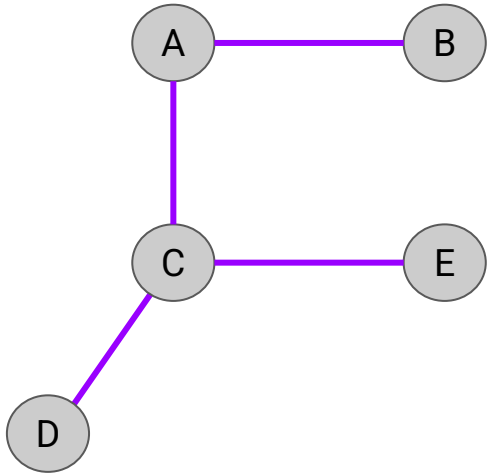
- Teoría de Grafos
- Algoritmo DFS
- Algoritmo BFS
- Prueba de Correctitud
- Complejidad Computacional
- Conclusiones



1. Teoría de Grafos

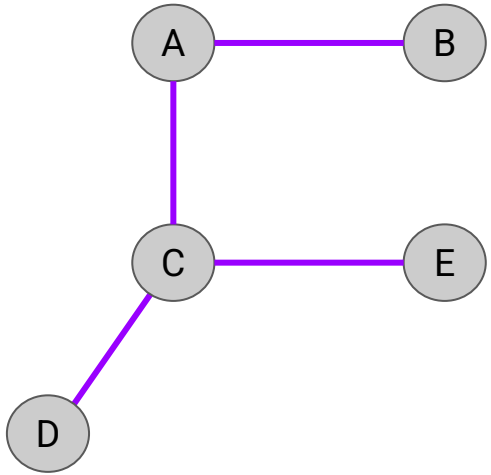
¿Qué es un grafo?

- Un grafo es una estructura de datos que consiste en un conjunto de vértices (nodos) conectados por aristas (enlaces).

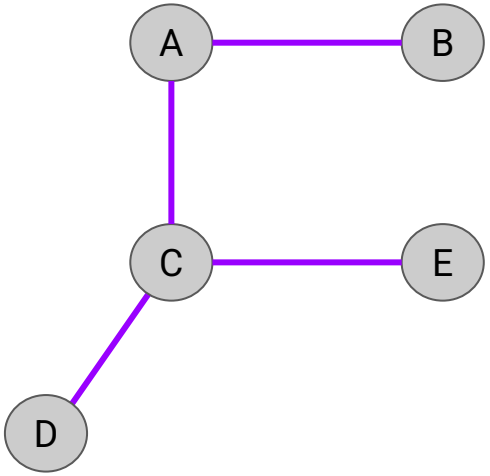


¿Qué es un grafo?

- Un grafo es una estructura de datos que consiste en un conjunto de vértices (nodos) conectados por aristas (enlaces).
- Los grafos son ampliamente utilizados para representar relaciones y conexiones en diversos problemas



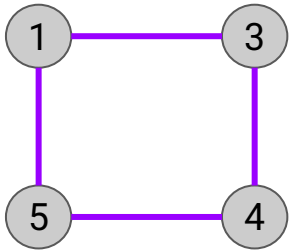
Definición



Un grafo es un conjunto de vértices (nodos) y aristas G=(V, E). Donde V es el conjunto de vértices y E el conjunto de aristas.

Tipos de grafos

Grafo Simple

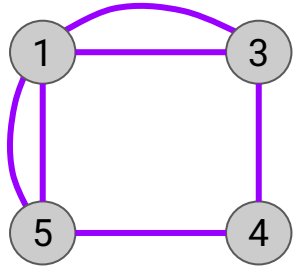


Aquel que acepta solo una arista para unir dos vértices

Tipos de grafos

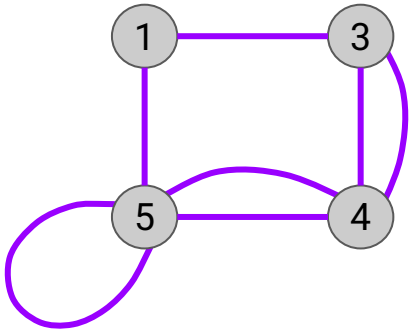
Aquel que acepta más de una arista para unir dos vértices

Multigrafo



Tipos de grafos

Pseudografo

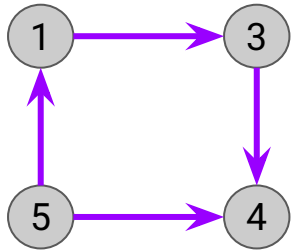


Aquel que acepta bucles

Tipos de grafos

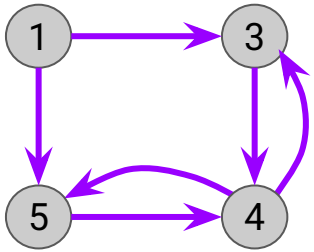
Aquel en que las aristas tienen dirección. Al mismo tiempo no acepta más de una arista entre vértices.

Grafo Dirigido



Tipos de grafos

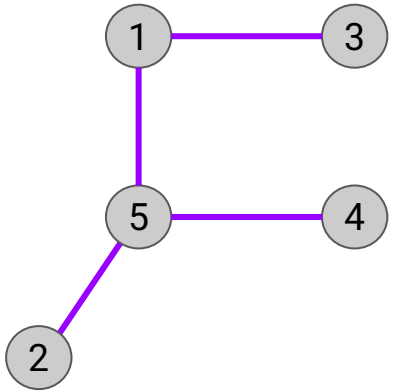
Multigrafo dirigido



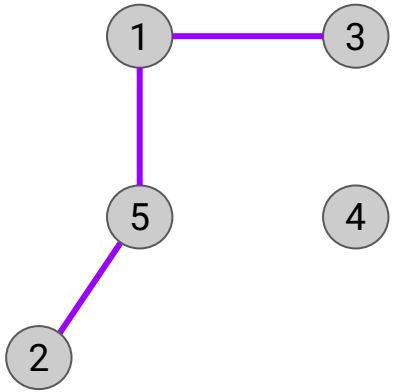
Aquel que acepta múltiples aristas dirigidas

Tipos de grafos

Grafo conexo



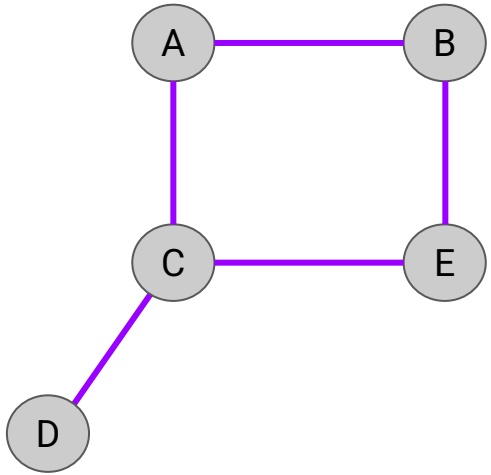
Grafo no conexo



Un grafo conexo es aquel en el que todos sus vértices están conectados por al menos una arista.

¿Por qué utilizar un grafo?

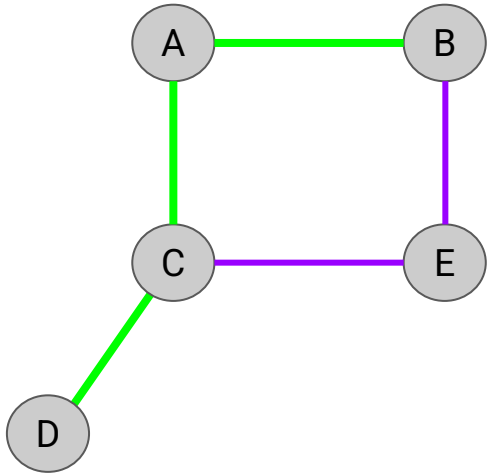
En la programación, a menudo necesitamos recorrer y explorar grafos para realizar diversas tareas, por ejemplo:



¿Por qué utilizar un grafo?

En la programación, a menudo necesitamos recorrer y explorar grafos para realizar diversas tareas, por ejemplo:

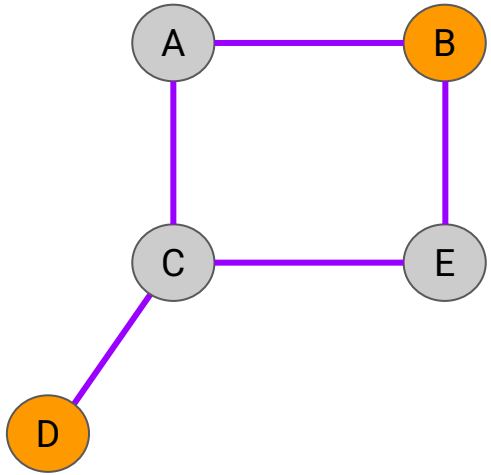
- Encontrar caminos



¿Por qué utilizar un grafo?

En la programación, a menudo necesitamos recorrer y explorar grafos para realizar diversas tareas, por ejemplo:

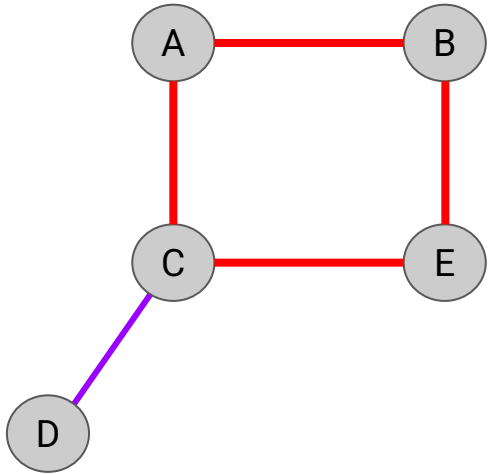
- Encontrar caminos
- Buscar componentes conectados



¿Por qué utilizar un grafo?

En la programación, a menudo necesitamos recorrer y explorar grafos para realizar diversas tareas, por ejemplo:

- Encontrar caminos
- Buscar componentes conectados
- Detectar ciclos y mucho más





2. Algoritmo DFS

Algoritmo DFS: Definición y Objetivos

- El algoritmo DFS (Depth-First Search) es una técnica de exploración de grafos que se enfoca en recorrer en **profundidad**.
- Su objetivo principal es explorar de manera exhaustiva todos los nodos alcanzables desde un nodo inicial.

Algoritmo DFS: Funcionamiento

- El algoritmo DFS comienza desde un nodo inicial y luego explora a profundidad todos los nodos adyacentes antes de retroceder.
- Utiliza una pila (stack) para realizar el seguimiento de los nodos a visitar.

Algoritmo DFS: Aplicaciones

- Detección de ciclos en un grafo.

Algoritmo DFS: Aplicaciones

- Detección de ciclos en un grafo.
- Componentes conectados en un grafo.

Algoritmo DFS: Aplicaciones

- Detección de ciclos en un grafo.
- Componentes conectados en un grafo.
- **Laberintos y búsqueda de caminos:** Para generar laberintos y también para encontrar el camino en un laberinto (el BFS es el más óptimo).

Algoritmo DFS: Aplicaciones

- Detección de ciclos en un grafo.
- Componentes conectados en un grafo.
- **Laberintos y búsqueda de caminos:** Para generar laberintos y también para encontrar el camino en un laberinto (el BFS es el más óptimo).
- **Árboles de expansión mínima:** El DFS se utiliza en algoritmos para encontrar árboles de expansión mínima, como el algoritmo de Prim.

Algoritmo DFS: Implementación (Iterativo)

```
1. DFS(nodo inicio):
2.   Crear una pila para los nodos visitados.
3.   Agregar el 'inicio' a la pila.
4.   Mientras la pila no esté vacía:
5.      Obtener el siguiente nodo (top) a visitar.
6.      Si el nodo actual no ha sido visitado:
7.          Marcar el nodo como visitado
8.          Procesar el nodo actual (imprimir)
9.          Para cada vecino no visitado:
10.             Agregar el vecino a la pila.
```

Algoritmo DFS: En C++

```
template <typename T>
void Grafo<T>::DFS(T u) {
  stack<T> s;
  s.push(u);
  unordered_set<T> visitados;
  while (!s.empty()) {
    T top = s.top();
    s.pop();
    if (visitados.find(top) == visitados.end()) {
      visitados.insert(top);
      cout << "Visitando: " << top << endl;
      for (const T &next : g_map[top]){
        if (visitados.find(next) == visitados.end())
          s.push(next);
      }
    }
  }
}
```

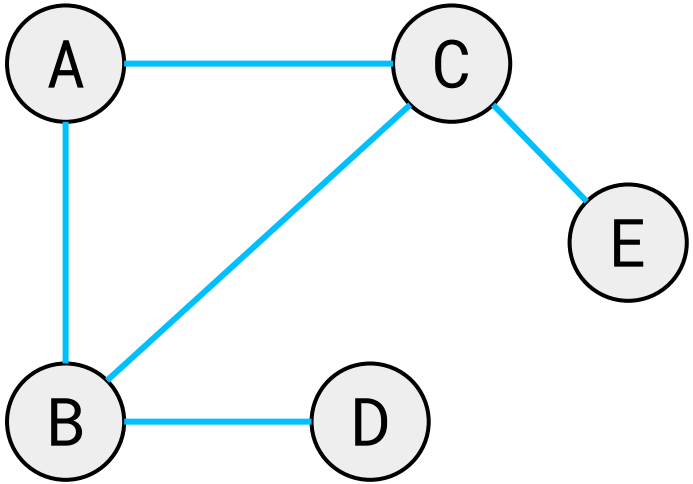
Algoritmo DFS: En C++

```
template<typename T>
struct Grafo{
    unordered_map<T,unordered_set<T>> g_map;

    void nueva_arista(T v1, T v2);
    void DFS(T u);
};
```

La clase **Grafo** tiene como atributo un **unordered_set** para los nodo visitado y un unordered_map para crear las conexiones entre nodos.

Algoritmo DFS: Ejemplo



Algoritmo DFS: Interfaz

```
int main(){
    Grafo<char> g;
    g.nueva_arista('A','B');
    g.nueva_arista('A','C');
    g.nueva_arista('B','C');
    g.nueva_arista('B','D');
    g.nueva_arista('C','E');

    g.DFS('A'); // A C E B D
}
```



3. Algoritmo BFS

Algoritmo BFS: Definición y Objetivos

- El algoritmo BFS (Breadth-first search) es otra técnica de exploración en grafos que se enfoca en recorrer en amplitud.
- Su objetivo principal es explorar los nodos vecinos antes de pasar a los siguientes niveles de nodos.

Algoritmo BFS: Funcionamiento

- El algoritmo BFS comienza desde un nodo inicial y explora todos los nodos vecinos antes de avanzar al siguiente nivel.
- Utiliza una cola (queue) para realizar el seguimiento de los nodos a visitar.

Algoritmo BFS: Aplicaciones

- **El camino más corto:** Encontrar el camino más corto entre dos nodos y explorar en niveles. Para grafos no ponderados.

Algoritmo BFS: Aplicaciones

- **El camino más corto:** Encontrar el camino más corto entre dos nodos y explorar en niveles. Para grafos no ponderados.
- **Árboles y estructuras de datos:** Para visualizar la estructura de un árbol o para realizar operaciones que requieran acceso a los nodos en el mismo nivel antes de descender a niveles inferiores.

Algoritmo BFS: Aplicaciones

- **El camino más corto:** Encontrar el camino más corto entre dos nodos y explorar en niveles. Para grafos no ponderados.
- **Árboles y estructuras de datos:** Para visualizar la estructura de un árbol o para realizar operaciones que requieran acceso a los nodos en el mismo nivel antes de descender a niveles inferiores.
- **Redes sociales:** Para la búsqueda de conexiones entre usuarios, la recomendación de amigos o contactos, y el análisis de comunidades o grupos.

Algoritmo BFS: Aplicaciones

- **El camino más corto:** Encontrar el camino más corto entre dos nodos y explorar en niveles. Para grafos no ponderados.
- **Árboles y estructuras de datos:** Para visualizar la estructura de un árbol o para realizar operaciones que requieran acceso a los nodos en el mismo nivel antes de descender a niveles inferiores.
- **Redes sociales:** Para la búsqueda de conexiones entre usuarios, la recomendación de amigos o contactos, y el análisis de comunidades o grupos.
- **Resolución de laberintos:** Cada celda del laberinto se considera un nodo y las conexiones entre las celdas vecinas se consideran aristas.

Algoritmo BFS: Implementación (Iterativo)

```
1.  BFS(nodo inicio):
2.    Crear una cola para visitar los nodos.
3.    Agregar el 'inicio' a la cola.
4.    Mientras la cola no esté vacía:
5.       Obtener el siguiente nodo (front) de la cola.
6.       Si el nodo actual no ha sido visitado:
7.           Marcar el nodo como visitado
8.           Procesar el nodo actual (imprimir)
9.           Obtener los vecinos del nodo actual.
10.          Para cada vecino no visitado:
11.              Agregar el vecino a la cola.
```

Algoritmo BFS: En C++

```
void Grafo<T>::BFS(T u){
    queue<T> Q;
    Q.push(u);
    unordered_set<T> visitados;

    while (Q.size() != 0){
        T v = Q.front();
        Q.pop();
        if (visitados.find(v) == visitados.end()){
            visitados.insert(v);
            cout << "Visitando -> " << v << endl;
            for (const T& e: g_map[v]){
                if (visitados.find(e) == visitados.end())
                    Q.push(e);
            }
        }
    }
}
```

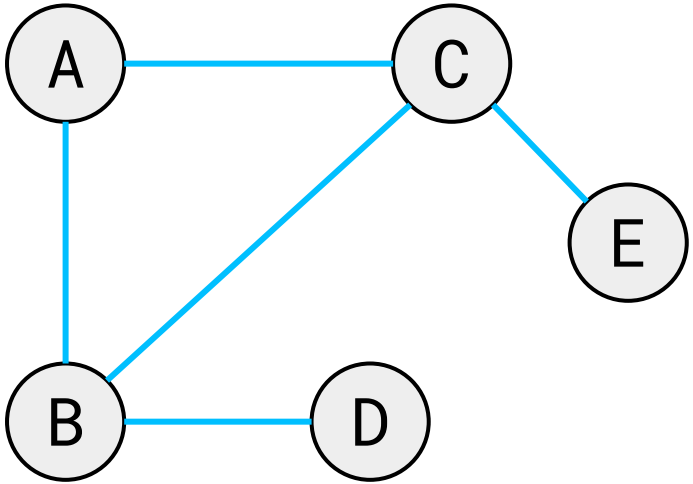
Algoritmo BFS: En C++

```
template<typename T>
struct Grafo{
    unordered_map<T,unordered_set<T>> g_map;

    void nueva_arista(T v1, T v2);
    void BFS(T u);
};
```

La clase **Grafo** tiene como atributo un **unordered_set** para los nodo visitado y un unordered_map para crear las conexiones entre nodos.

Algoritmo BFS: Ejemplo



Algoritmo BFS: Interfaz

```
int main(){
    Grafo<char> g;
    g.nueva_arista('A','B');
    g.nueva_arista('A','C');
    g.nueva_arista('B','C');
    g.nueva_arista('B','D');
    g.nueva_arista('C','E');

    g.BFS('A'); // Salida: A C B E D
}
```

4. Prueba de Correctitud

DFS: Prueba de Correctitud

Método Inductivo:

- Para probar la correctitud del algoritmo DFS, supongamos que en **cada etapa(llamada) del algoritmo se han visitado todos los nodos alcanzables desde el nodo inicial hasta el actual.**
- Es decir, todos los nodos que son accesibles desde el nodo inicial se visitan en algún momento por el algoritmo DFS.

DFS: Prueba de Correctitud

Caso Base:

- En el caso base del algoritmo se comienza con el nodo **inicial**.
- En este caso, la hipótesis inductiva se cumple trivialmente, ya que el nodo inicial se visita inmediatamente.

DFS: Prueba de Correctitud

Paso Inductivo:

- Supongamos que la hipótesis inductiva es verdadera en un paso/llamada dado del algoritmo DFS.

DFS: Prueba de Correctitud

Paso Inductivo:

- Supongamos que la hipótesis inductiva es verdadera en un paso/llamada dado del algoritmo DFS.
- Ahora, consideremos el siguiente paso, donde se elige un nodo vecino no visitado para continuar la exploración.

DFS: Prueba de Correctitud

Paso Inductivo:

- Supongamos que la hipótesis inductiva es verdadera en un paso/llamada dado del algoritmo DFS.
- Ahora, consideremos el siguiente paso, donde se elige un nodo vecino no visitado para continuar la exploración.
- Por la hipótesis inductiva, **todos los nodos alcanzables desde el nodo inicial hasta el paso actual ya se han visitado**.

DFS: Prueba de Correctitud

Paso Inductivo:

- Supongamos que la hipótesis inductiva es verdadera en un paso/llamada dado del algoritmo DFS.
- Ahora, consideremos el siguiente paso, donde se elige un nodo vecino no visitado para continuar la exploración.
- Por la hipótesis inductiva, **todos los nodos alcanzables desde el nodo inicial hasta el paso actual ya se han visitado**.
- Por lo tanto, al visitar el nodo vecino, **se están visitando todos los nodos alcanzables hasta el nodo vecino**.

DFS: Prueba de Correctitud

Demostración:

- Dado que la hipótesis inductiva se cumple, todos los nodos alcanzables desde el nodo inicial hasta el paso actual ya se han visitado.
- Al visitar un nuevo nodo, la hipótesis inductiva se mantiene para los nodos alcanzables desde ese nodo.
- Por lo tanto, **el algoritmo termina cuando todos los nodos alcanzables se visitan**.

BFS: Prueba de Correctitud

Método Inductivo:

- Para probar la correctitud del BFS, supongamos que el algoritmo **visita los nodos en orden a su distancia al nodo inicial**.
- Es decir, si un nodo A está en la cola antes que un nodo B, entonces la distancia de A al nodo inicial es menor o igual que la distancia de B al nodo inicial.

BFS: Prueba de Correctitud

Caso Base:

- Comencemos con el caso base del algoritmo BFS, donde el nodo inicial se agrega a la cola.
- En este caso, la hipótesis inductiva se cumple trivialmente, ya que solo hay un nodo en la cola y su distancia al nodo inicial es 0.

BFS: Prueba de Correctitud

Paso Inductivo:

- Supongamos que la hipótesis inductiva es verdadera en un paso o llamada al algoritmo BFS.

BFS: Prueba de Correctitud

Paso Inductivo:

- Supongamos que la hipótesis inductiva es verdadera en un paso o llamada al algoritmo BFS.
- Ahora, consideremos el siguiente paso, donde se extrae un nodo de la cola y se agregan sus vecinos no visitados.

BFS: Prueba de Correctitud

Paso Inductivo:

- Supongamos que la hipótesis inductiva es verdadera en un paso o llamada al algoritmo BFS.
- Ahora, consideremos el siguiente paso, donde se extrae un nodo de la cola y se agregan sus vecinos no visitados.
- Por la hipótesis inductiva, **los vecinos se agregan a la cola en orden de su distancia al nodo inicial**.

BFS: Prueba de Correctitud

Demostración:

- Dado que los vecinos se agregan en orden de su distancia al nodo inicial, **la hipótesis inductiva se mantiene para los nuevos nodos agregados**.
- Por lo tanto, el algoritmo termina cuando todos los nodos fueron explorados, **pero en orden de su distancia al nodo inicial**.



5. Complejidad Computacional

DFS: Búsqueda en Profundidad

- El algoritmo DFS recorre un grafo explorando lo más profundo posible antes de retroceder. Ej: Es como si estuviéramos explorando un laberinto siguiendo un camino hasta que no haya más opciones y luego retrocediendo para explorar otros caminos.

DFS: Búsqueda en Profundidad

- El algoritmo DFS recorre un grafo explorando lo más profundo posible antes de retroceder. Ej: Es como si estuviéramos explorando un laberinto siguiendo un camino hasta que no haya más opciones y luego retrocediendo para explorar otros caminos.
- En el peor de los casos, el algoritmo DFS visita cada nodo y cada arista del grafo una vez. Entonces, el tiempo que tarda en ejecutarse depende del número de nodos y aristas del grafo.

DFS: Búsqueda en Profundidad

- El algoritmo DFS recorre un grafo explorando lo más profundo posible antes de retroceder. Ej: Es como si estuviéramos explorando un laberinto siguiendo un camino hasta que no haya más opciones y luego retrocediendo para explorar otros caminos.
- En el peor de los casos, el algoritmo DFS visita cada nodo y cada arista del grafo una vez. Entonces, el tiempo que tarda en ejecutarse depende del número de nodos y aristas del grafo.
- Si el grafo tiene **V** nodos (vértices) y **E** aristas, la complejidad de tiempo del DFS es de **O(V + E)**. Esto significa que el tiempo de ejecución crece proporcionalmente al número de nodos y aristas.

BFS: Búsqueda en Anchura

- El algoritmo BFS también recorre un grafo, pero lo hace **en forma de capas**. Comienza explorando todos los nodos vecinos al nodo de partida, luego los vecinos de esos nodos, y así sucesivamente. Ej: Es como si estuviéramos explorando un laberinto en niveles, primero los que están más cerca y luego los más lejanos.

BFS: Búsqueda en Anchura

- El algoritmo BFS también recorre un grafo, pero lo hace **en forma de capas**. Comienza explorando todos los nodos vecinos al nodo de partida, luego los vecinos de esos nodos, y así sucesivamente. Ej: Es como si estuviéramos explorando un laberinto en niveles, primero los que están más cerca y luego los más lejanos.
- En el peor de los casos, el algoritmo BFS también visita cada nodo y cada arista del grafo una vez. Por lo tanto, su complejidad de tiempo también es de O(V + E), donde V es el número de nodos y E es el número de aristas

Al grafo puede determinar la complejidad

- Si el grafo es disperso y contiene pocos enlaces entre nodos, la cantidad de aristas (**E**) puede ser mucho menor que el número de nodos (**V**), lo que resulta en una complejidad de tiempo cercana a $\mathbf{O(V)}$.

Al grafo puede determinar la complejidad

- Si el grafo es disperso y contiene pocos enlaces entre nodos, la cantidad de aristas (**E**) puede ser mucho menor que el número de nodos (**V**), lo que resulta en una complejidad de tiempo cercana a **O(V)**.
- Por otro lado, si el grafo es denso y cada nodo está conectado con muchos otros nodos, la cantidad de aristas (**E**) puede acercarse a **V^2**, lo que resulta en una complejidad de tiempo cercana a **O(V^2)**.

6. Conclusiones

Conclusiones

- BFS es un algoritmo que explora el grafo nivel por nivel, comenzando desde un nodo de origen y avanzando hacia los nodos adyacentes. Útil para encontrar el camino más corto entre dos nodos y para buscar componentes conexos en un grafo no dirigido.
- DFS es un algoritmo que explora el grafo en profundidad, siguiendo un camino específico hasta alcanzar un nodo final antes de retroceder y explorar otros caminos. Útil para recorrer todos los nodos de un grafo, encontrar ciclos.
- Tanto BFS como DFS tienen una complejidad de tiempo de $\mathbf{O(V+E)}$, donde $\mathbf{V}$ es el número de nodos y $\mathbf{E}$ es el número de aristas.



Resumen

En esta sesión se trataron los tópicos siguientes:

- Algoritmos DFS y BFS